

UDP-Based File Transfer System Similar to TFTP (with Enhanced Features)

General Description:

In this exercise, you are tasked with developing a file transfer system over a network using the UDP protocol in C or C++, similar to TFTP (Trivial File Transfer Protocol), but with enhanced support for more efficient and secure file transfers. The system will support uploading, downloading, and deleting files, similar to TFTP, and will include features such as packet acknowledgment, file backup and recovery, and improved security.

Improvements Over TFTP:

1. **Improved Packet Management:** Increase the size of transferred packets and add a mechanism for resending lost packets.
2. **Backup and File Recovery:** Every file uploaded to the server will be backed up, allowing recovery in case of failure.
3. **Security:** Implement encryption for file transfers between the client and server.
4. **Full TFTP Support:** The system will be compatible with standard TFTP clients.

Technical Requirements:

1. **UDP Protocol:** The system will use the UDP protocol to transfer files between the client and server, similar to TFTP. Since UDP is faster but less reliable, the system will handle packet loss by using an acknowledgment mechanism.
2. **File Transfer Management:**
 - **File Upload:** The client will send the file to the server in small packets, and the server will acknowledge each packet before the next one is sent.
 - **File Download:** The client will request a file from the server, and the server will send the file in acknowledged packets.
 - **File Deletion:** The client can send a request to delete a file from the server.
3. **Reliability Enhancements:**
 - **Acknowledgment Packets (ACK):** For each packet received, the server will send an acknowledgment (ACK) to the client. If the client doesn't receive an ACK, it will resend the packet.
 - **Packet Loss Handling:** If the client does not receive a packet in time, it will request a retransmission from the server.
4. **File Backup and Recovery:**
 - The server will store a backup of every file uploaded to it.
 - In the event of a crash or data loss, files can be recovered from the backup.
5. **Security:**
 - **Encryption:** Every file transferred between the client and server will be encrypted using AES to ensure secure communication.
 - **File Integrity Check:** The client will be able to verify file integrity using a checksum or hash (e.g., MD5).

Structure of the Exercise:

Part 1: Creating an Enhanced UDP Server

1. **Listening for Requests:** The server will listen for client requests on a UDP port and accept requests to upload, download, or delete files.
2. **Packet Acknowledgment:** After receiving each packet, the server will send an acknowledgment (ACK) back to the client. If the client does not receive an ACK, it will resend the packet.
3. **File Management:** The server will store files uploaded by clients in a dedicated directory. It will also be able to delete files as requested by the client.

Part 2: Creating an Enhanced UDP Client

1. **Sending Requests:** The client will be able to send requests to the server to upload, download, or delete files.
2. **Uploading/Downloading in Packets:** The client will send or receive files in small chunks (packets). Each successfully received packet will be acknowledged by the server.
3. **Reliable Communication:** If a packet is lost, the client will request the server to retransmit it.

Part 3: TFTP Compatibility

1. **TFTP Protocol Support:** The system will use a packet format compatible with TFTP so that it can be accessed by standard TFTP clients.
2. **Handling Standard TFTP Requests:** The server will be able to handle standard TFTP requests such as Read Request (RRQ), Write Request (WRQ), and Error messages.

UDP Packet Structure:

- **Requests:**
 - Read Request (RRQ): Request to download a file.
 - Write Request (WRQ): Request to upload a file.
 - Delete Request: Request to delete a file.
- **Packet Contents:**
 - Operation ID (upload/download/delete).
 - Filename.
 - File data sent in chunks.
- **Packet Acknowledgment:**
 - Every packet received will be acknowledged with an ACK, allowing the client to continue sending the next packet.

Example Workflow:

1. **Client 1** sends a Read Request (RRQ) for the file "file.txt" to the server.

2. The server acknowledges the request and starts sending the file in packets of a defined size (e.g., 512 bytes per packet).
3. Each packet received by the client is checked and acknowledged by the server. If a packet is lost, the client will request a retransmission.
4. Once the transfer is complete, the client will have received the entire file.

File Structure:

- **server.c:** Contains the server logic – listening for UDP requests, managing files, and acknowledging packets.
- **client.c:** Contains the client logic – sending requests, receiving packets, and uploading/downloading files.
- **udp_file_transfer.h:** Header file defining packet formats, necessary functions for uploading and downloading, and packet acknowledgment.

Possible Additions (Advanced Features):

1. **Version Control:** Add support for versioning each file. The client can request a specific version of a file or maintain multiple versions on the server.
2. **Server-to-Server File Synchronization:** Implement a mechanism for two servers to synchronize files between each other so that if one server fails, the other can recover the data.
3. **Multithreading Support:** Add multithreading support to the server to handle multiple connections simultaneously.

Good luck!